DAY-4 ASSIGNMENT [DSA-PREFIX]

UDAY CHIRRA
2303A510G2

1. Running Sum of 1d Array

```
class Solution {
    public int[] runningSum(int[] nums) {
        for (int i = 1; i < nums.length; i++) {
            nums[i] += nums[i - 1];
        }
        return nums;
    }
}
```

2. Range Sum Query - Immutable

```
class NumArray {
    private int[] prefix;

    public NumArray(int[] nums) {
        prefix = new int[nums.length + 1];

        for (int i = 0; i < nums.length; i++) {
            prefix[i + 1] = prefix[i] + nums[i];
        }
    }

    public int sumRange(int left, int right) {
        return prefix[right + 1] - prefix[left];
    }
}
```

3. Find the Middle Index in Array

```
class Solution {
    public int findMiddleIndex(int[] nums) {
        int total = 0;

        for (int num : nums) {
            total += num;
```

```
        }

        int leftSum = 0;

        for (int i = 0; i < nums.length; i++) {
            int rightSum = total - leftSum - nums[i];

            if (leftSum == rightSum) {
                return i;
            }

            leftSum += nums[i];
        }

        return -1;
    }
}
```

4. Find Pivot Index

```
class Solution {
    public int pivotIndex(int[] nums) {
        int total = 0;

        for (int num : nums) {
            total += num;
        }

        int leftSum = 0;

        for (int i = 0; i < nums.length; i++) {
            if (leftSum == total - leftSum - nums[i]) {
                return i;
            }

            leftSum += nums[i];
        }

        return -1;
    }
}
```

5. Subarray Sum Equals K

```
class Solution {
    public int subarraySum(int[] nums, int k) {
        HashMap<Integer, Integer> map = new HashMap<>();
        map.put(0, 1);

        int sum = 0;
        int count = 0;

        for (int num : nums) {
            sum += num;

            if (map.containsKey(sum - k)) {
                count += map.get(sum - k);
            }

            map.put(sum, map.getOrDefault(sum, 0) + 1);
        }

        return count;
    }
}
```

6. Contiguous Array

```
class Solution {
    public int findMaxLength(int[] nums) {
        HashMap<Integer, Integer> map = new HashMap<>();
        map.put(0, -1);

        int count = 0;
        int maxLength = 0;

        for (int i = 0; i < nums.length; i++) {
            if (nums[i] == 0) {
                count--;
            } else {
                count++;
            }
```

```
            if (map.containsKey(count)) {
                maxLength = Math.max(
                    maxLength,
                    i - map.get(count)
                );
            } else {
                map.put(count, i);
            }
        }

        return maxLength;
    }
}
```

7. Continuous Subarray Sum

```
class Solution {
    public boolean checkSubarraySum(int[] nums, int k) {
        HashMap<Integer, Integer> map = new HashMap<>();
        map.put(0, -1);

        int sum = 0;

        for (int i = 0; i < nums.length; i++) {
            sum += nums[i];

            int remainder = sum % k;

            if (remainder < 0) {
                remainder += k;
            }

            if (map.containsKey(remainder)) {
                if (i - map.get(remainder) >= 2) {
                    return true;
                }
            } else {
                map.put(remainder, i);
            }
        }
```

```
        return false;
    }
}
```

8. Matrix Block Sum

```
class Solution {
    public int[][] matrixBlockSum(int[][] mat, int k) {
        int m = mat.length;
        int n = mat[0].length;

        int[][] prefix = new int[m + 1][n + 1];

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                prefix[i][j] =
                    mat[i - 1][j - 1]
                    + prefix[i - 1][j]
                    + prefix[i][j - 1]
                    - prefix[i - 1][j - 1];
            }
        }

        int[][] result = new int[m][n];

        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {

                int r1 = Math.max(0, i - k);
                int c1 = Math.max(0, j - k);
                int r2 = Math.min(m - 1, i + k);
                int c2 = Math.min(n - 1, j + k);

                result[i][j] =
                    prefix[r2 + 1][c2 + 1]
                    - prefix[r1][c2 + 1]
                    - prefix[r2 + 1][c1]
                    + prefix[r1][c1];
            }
        }
```

```
        return result;
    }
}
```

9. Number of Submatrices That Sum to Target

```
class Solution {
    public int numSubmatrixSumTarget(int[][] matrix, int target) {
        int rows = matrix.length;
        int cols = matrix[0].length;
        int count = 0;

        for (int top = 0; top < rows; top++) {
            int[] sum = new int[cols];

            for (int bottom = top; bottom < rows; bottom++) {

                for (int col = 0; col < cols; col++) {
                    sum[col] += matrix[bottom][col];
                }

                HashMap<Integer, Integer> map = new HashMap<>();
                map.put(0, 1);

                int prefix = 0;

                for (int value : sum) {
                    prefix += value;

                    count += map.getOrDefault(
                        prefix - target,
                        0
                    );

                    map.put(
                        prefix,
                        map.getOrDefault(prefix, 0) + 1
                    );
                }
            }
        }
```

```
        return count;
    }
}
```

10. Count of Range Sum

```
class Solution {
    public int countRangeSum(int[] nums, int lower, int upper) {
        long[] prefix = new long[nums.length + 1];

        for (int i = 0; i < nums.length; i++) {
            prefix[i + 1] = prefix[i] + nums[i];
        }

        return mergeSort(prefix, 0, prefix.length, lower, upper);
    }

    private int mergeSort(
        long[] nums,
        int left,
        int right,
        int lower,
        int upper
    ) {
        if (right - left <= 1) {
            return 0;
        }

        int mid = left + (right - left) / 2;

        int count =
            mergeSort(nums, left, mid, lower, upper)
            + mergeSort(nums, mid, right, lower, upper);

        int low = mid;
        int high = mid;

        for (int i = left; i < mid; i++) {

            while (low < right &&
```

```
                nums[low] - nums[i] < lower) {
            low++;
        }

        while (high < right &&
                nums[high] - nums[i] <= upper) {
            high++;
        }

        count += high - low;
    }

    long[] temp = new long[right - left];
    int i = left;
    int j = mid;
    int k = 0;

    while (i < mid && j < right) {
        if (nums[i] <= nums[j]) {
            temp[k++] = nums[i++];
        } else {
            temp[k++] = nums[j++];
        }
    }

    while (i < mid) {
        temp[k++] = nums[i++];
    }

    while (j < right) {
        temp[k++] = nums[j++];
    }

    System.arraycopy(temp, 0, nums, left, temp.length);

    return count;
  }
}
```